LAB MANUAL

Name: V.Madhulika
Register Name: 192312620
Course Code: CSA1709

Exp :01
8-Puzzle Problem

Aim:

To write a python program to solve the 8-puzzle problem.

Objective:

To understand heuristic-based search techniques and apply the A* algorithm with the Manhattan distance heuristic to solve the 8-puzzle sliding tile problem.

Algorithm:

1. Dequeue the state with the lowest priority from the frontier.
2. If the dequeued state is the goal state, return the solution path.
3. Add the dequeued state to the explored set.
4. Generate all possible successor states by swapping the empty tile with its neighboring tiles.
5. For each successor state, calculate its priority based on the Manhattan distance and add it to the frontier if it has not been explored before.

Program

```
from queue import PriorityQueue

goal = (1, 2, 3, 4, 5, 6, 7, 8, 0)

def heuristic(state):
    h = 0
    for i in range(9):
        if state[i] != 0:
            x1, y1 = divmod(i, 3)
            x2, y2 = divmod(state[i]-1, 3)
            h += abs(x1-x2) + abs(y1-y2)
    return h

def successors(state):
    moves = []
    i = state.index(0)
    r, c = divmod(i, 3)
    directions = [(-1,0),(1,0),(0,-1),(0,1)]
    for dr, dc in directions:
        nr, nc = r+dr, c+dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            ni = nr*3 + nc
            new = list(state)
            new[i], new[ni] = new[ni], new[i]
            moves.append(tuple(new))
    return moves

def solve(start):
    pq = PriorityQueue()
    pq.put((heuristic(start), 0, start, [start]))
```

```
    visited = set()
    while not pq.empty():
        f, g, state, path = pq.get()
        if state == goal:
            return path
        if state in visited:
            continue
        visited.add(state)
        for nxt in successors(state):
            if nxt not in visited:
                pq.put((g+1+heuristic(nxt), g+1, nxt, path+[nxt]))
    return None

start = (1, 2, 3, 5, 6, 0, 7, 8, 4)
solution = solve(start)
if solution:
    print("Solution Path:\n")
    for state in solution:
        for i in range(0, 9, 3):
            print(state[i], state[i+1], state[i+2])
        print()
else:
    print("No Solution")
```

Output

```
1 2 3
5 6 0
7 8 4

1 2 3
5 0 6
7 8 4

1 2 3
0 5 6
7 8 4

1 2 3
7 5 6
0 8 4

1 2 3
7 5 6
8 0 4

1 2 3
7 5 6
8 4 0

1 2 3
7 5 0
8 4 6
```

Result

Hence, the simple python program for 8- puzzle is done

Exp: 02
Write the Python Program to solve 8-Queen Problem

Aim:

To write a python program to solve 8-Queen problem.

Objective:

To understand the backtracking search technique by placing 8 queens on an 8x8 chessboard such that no two queens attack each other.

Algorithm:

1. Start with an empty board of size 8x8.
2. Place the first queen in the first row and the first column.
3. For each subsequent row, check all possible columns to place the queen, and backtrack if a valid position cannot be found.
4. A valid position is a column where no other queens are in the same row, column, or diagonal.
5. If no solution is found, return failure

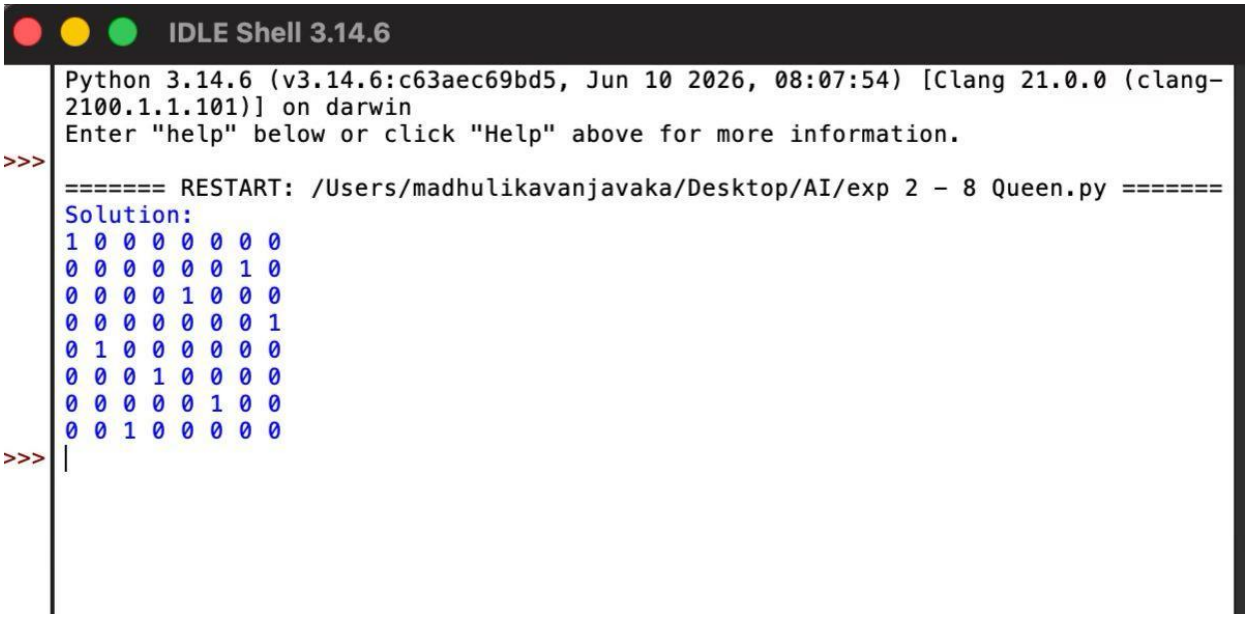
Program

```
N = 8
board = [[0] * N for _ in range(N)]

def is_safe(row, col):
    for i in range(col):
        if board[row][i]:
            return False
    i, j = row, col
    while i >= 0 and j >= 0:
        if board[i][j]:
            return False
        i -= 1
        j -= 1
    i, j = row, col
    while i < N and j >= 0:
        if board[i][j]:
            return False
        i += 1
        j -= 1
    return True

def solve(col):
    if col >= N:
        return True
    for row in range(N):
        if is_safe(row, col):
            board[row][col] = 1
            if solve(col + 1):
                return True
            board[row][col] = 0
```

```
    return False

if solve(0):
    print("Solution:")
    for row in board:
        print(*row)
else:
    print("No solution exists")
```

Output

```
1 0 0 0 0 0 0 0
0 0 0 0 0 0 1 0
0 0 0 0 1 0 0 0
0 0 0 0 0 0 0 1
0 1 0 0 0 0 0 0
0 0 0 1 0 0 0 0
0 0 0 0 0 1 0 0
0 0 1 0 0 0 0 0
```



Result

Hence, the simple python program for 8- Queen problem is done

Exp:03
Write the Python Program for Water Jug Problem

Aim:

To write a python program for water jug problem

Objective:

To understand state-space search by finding a sequence of operations (fill, empty, pour) that measures a target quantity of water using two jugs of different capacities.

Algorithm:

1. Start with two empty jugs of different capacities, and a target quantity of water to measure.
2. Fill one jug to its maximum capacity.
3. Pour the water from the full jug into the other jug until it is full or the first jug is empty.
4. If the second jug is full, pour out the water to empty it.
5. Repeat steps 2-4, filling and pouring water between the two jugs, until the desired quantity of water is measured or all possible combinations are tried.

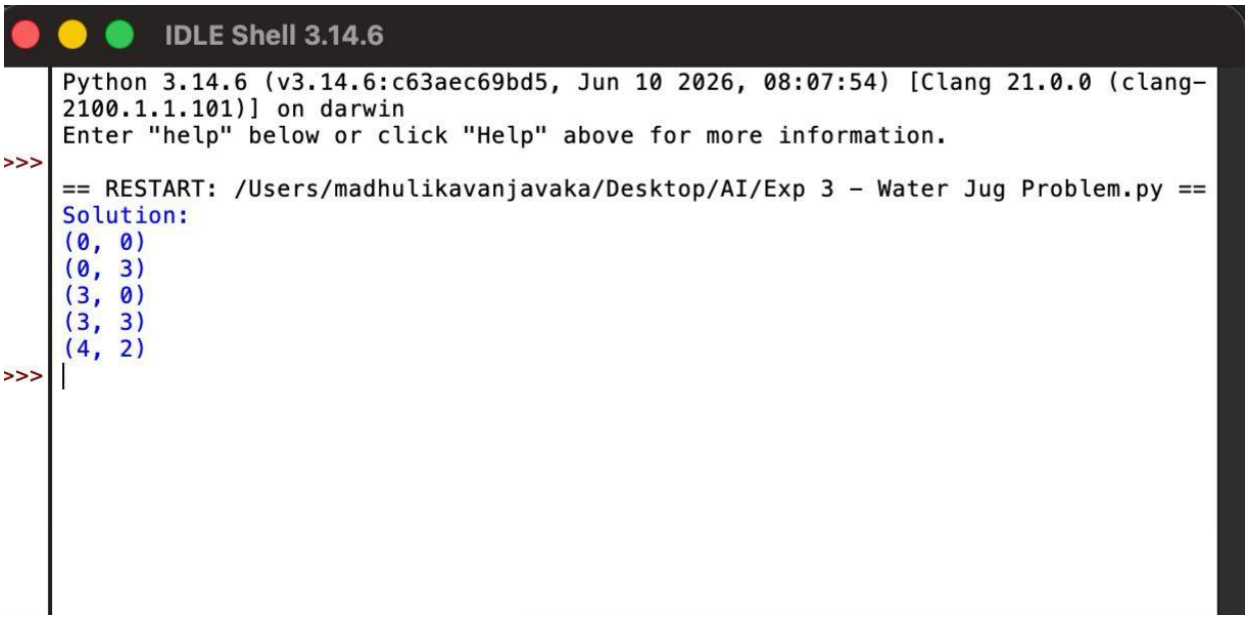
Program

```
from collections import deque

def water_jug():
    visited = set()
    queue = deque()
    start = (0, 0)
    goal = 2
    queue.append((start, [start]))
    visited.add(start)
    while queue:
        (a, b), path = queue.popleft()
        if a == goal or b == goal:
            return path
        next_states = [
            (4, b),                         # Fill 4L jug
            (a, 3),                         # Fill 3L jug
            (0, b),                         # Empty 4L jug
            (a, 0),                         # Empty 3L jug
            (max(0, a-(3-b)), min(3, b+a)),     # Pour 4L -> 3L
            (min(4, a+b), max(0, b-(4-a)))      # Pour 3L -> 4L
        ]
        for state in next_states:
            if state not in visited:
                visited.add(state)
                queue.append((state, path + [state]))

path = water_jug()
print("Solution:")
for state in path:
```

```
print(state)
```

Output

(0, 0)
(0, 3)
(3, 0)
(3, 3)
(4, 2)



Result

Hence, the simple python program for water jug problem is done

Exp: 04
Write the Python Program for Cript-Arithmetic Problem.

Aim:

To write a python program for Cript-Arithematic problem.

Objective:

To understand constraint satisfaction problems (CSP) by assigning unique digits to letters such that the resulting arithmetic equation holds true.

Algorithm:

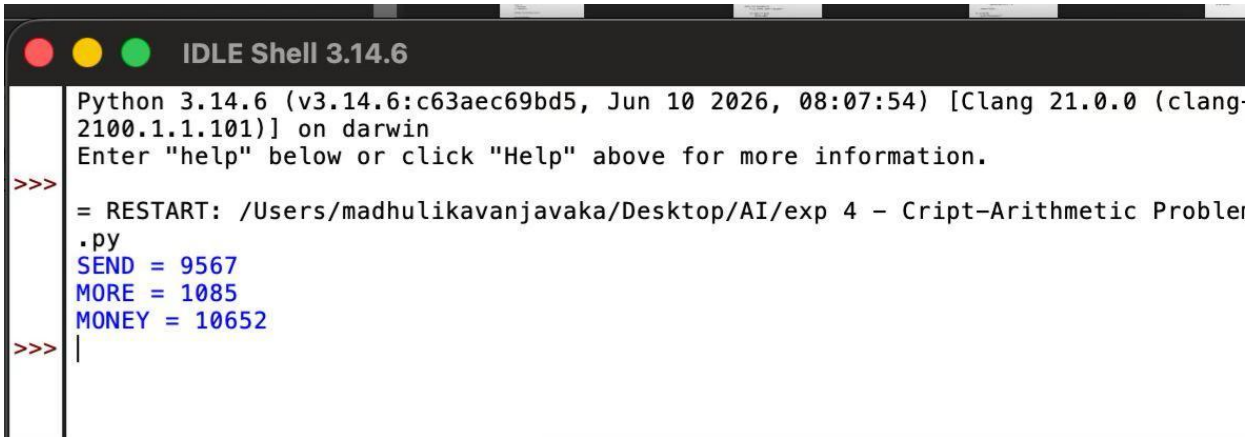
1. Write down the arithmetic problem in full, including any carry-overs.
2. Replace each letter with a unique digit from 0 to 9.
3. Generate all possible combinations of digits for each letter, subject to any constraints from the carry-overs and the rules of arithmetic.
4. For each combination of digits, evaluate the arithmetic problem and check if it satisfies the given constraints.
5. If a solution is found, output the values of the letters and the arithmetic problem, and exit the algorithm.
6. If no solution is found, output a message indicating that no solution exists.

Program

```
from itertools import permutations

letters = ('S', 'E', 'N', 'D', 'M', 'O', 'R', 'Y')
digits = (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)

for p in permutations(digits, 8):
    S, E, N, D, M, O, R, Y = p
    if S == 0 or M == 0:
        continue
    SEND = 1000*S + 100*E + 10*N + D
    MORE = 1000*M + 100*O + 10*R + E
    MONEY = 10000*M + 1000*O + 100*N + 10*E + Y

    if SEND + MORE == MONEY:
        print("SEND =", SEND)
        print("MORE =", MORE)
        print("MONEY =", MONEY)
        break
```

Output

```
SEND = 9567
MORE = 1085
MONEY = 10652
```



Result

The output has verified.

Exp :05
Write the python program for Missionaries Cannibal problem.

Aim:

To write a python program for Missionaries Cannibal problem.

Objective:

To understand state-space search with constraints by transporting 3 missionaries and 3 cannibals across a river using a boat, without cannibals ever outnumbering missionaries on either bank.

Algorithm:

1. Create a data structure to represent the state of the problem, including the number of missionaries and cannibals on each side of the river and the position of the boat.
2. Create a data structure to represent the state space of the problem, including the initial state, the goal state, and the possible transitions between states.
3. Use a search algorithm, such as depth-first search or breadth-first search, to explore the state space and find a solution.
4. At each step of the search, generate all possible successor states from the current state by applying one of the allowed boat movements (two missionaries, two cannibals, or one of each).
5. Check if the successor state is valid (i.e., no more cannibals than missionaries on either side of the river).
6. If the successor state is valid, add it to the search tree and continue the search.
7. If the successor state is the goal state, return the path from the initial state to the goal state.
8. If there are no more possible successor states to explore, backtrack to the previous state and try another possible movement.

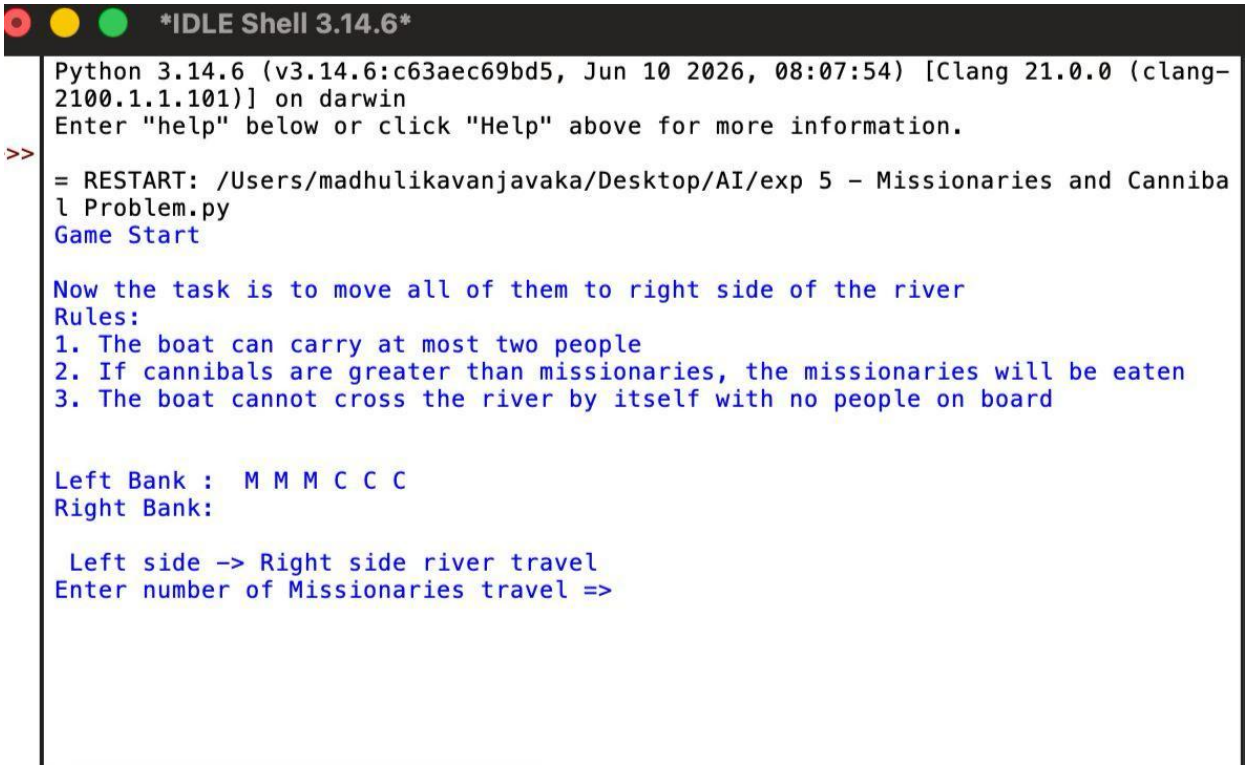
Program

```
left_m = 3
left_c = 3
right_m = 0
right_c = 0
boat = "left"

print("Game Start\n")
print("Now the task is to move all of them to right side of the river")
print("Rules:")
print("1. The boat can carry at most two people")
print("2. If cannibals are greater than missionaries, the missionaries will be eaten")
print("3. The boat cannot cross the river by itself with no people on board\n")

while True:
   print("\nLeft Bank : ", "M " * left_m + "C " * left_c)
   print("Right Bank:", "M " * right_m + "C " * right_c)
```

```
    if left_m == 0 and left_c == 0:
        print("\nCongratulations! You solved the problem.")
        break

    print("\n", boat.capitalize(), "side ->", "Right" if boat == "left" else "Left", "side river travel")
    m = int(input("Enter number of Missionaries travel => "))
    c = int(input("Enter number of Cannibals travel => "))

    if m + c == 0 or m + c > 2:
        print("Invalid input please retry !!")
        continue

    if boat == "left":
        if m > left_m or c > left_c:
            print("Invalid input please retry !!")
            continue
        left_m -= m
        left_c -= c
        right_m += m
        right_c += c
        boat = "right"
    else:
        if m > right_m or c > right_c:
            print("Invalid input please retry !!")
            continue
        right_m -= m
        right_c -= c
        left_m += m
        left_c += c
        boat = "left"

    if (left_m > 0 and left_c > left_m) or (right_m > 0 and right_c > right_m):
        print("\nCannibals ate the Missionaries!")
        print("Game Over.")
        break
```

Output

```
Game Start

Now the task is to move all of them to right side of the river
Rules:
1. The boat can carry at most two people
2. If cannibals are greater than missionaries, the missionaries will be eaten
3. The boat cannot cross the river by itself with no people on board

Left Bank :  M M M C C C
Right Bank:

 Left side -> Right side river travel
Enter number of Missionaries travel =>
```



Result

The program was executed and the output was found to be correct

Exp: 06
Write the python program for Vacuum Cleaner Problem.

Aim:

To write a python program for vacuum cleaner problem.

Objective:

To understand the concept of a simple reflex agent that perceives its environment (room status) and acts (suck/move) based on condition-action rules.

Algorithm:

1. Create a data structure to represent the state of the problem, including the current position of the vacuum cleaner and the state of each square in the room (clean or dirty).
2. Create a data structure to represent the state space of the problem, including the initial state, the goal state, and the possible transitions between states.
3. Use a search algorithm, such as depth-first search or breadth-first search, to explore the state space and find a solution.
4. At each step of the search, generate all possible successor states from the current state by applying one of the allowed actions (move up, down, left, right, or clean).
5. Check if the successor state is valid (i.e., the vacuum cleaner cannot move outside the room, and it can only clean dirty squares).
6. If the successor state is valid, add it to the search tree and continue the search.
7. If the successor state is the goal state, return the path from the initial state to the goal state.
8. If there are no more possible successor states to explore, backtrack to the previous state and try another possible action.

Program

```
def vacuum_cleaner():
    rooms = {'A': 'Dirty', 'B': 'Dirty'}
    current = 'A'
    while True:
        print("\nCurrent Location:", current)
        print("Room Status:", rooms)
        if rooms[current] == 'Dirty':
            print("Action: Clean")
            rooms[current] = 'Clean'
        else:
            print("Action: Move")
            if current == 'A':
                current = 'B'
            else:
                current = 'A'
        if rooms['A'] == 'Clean' and rooms['B'] == 'Clean':
            print("\nBoth rooms are clean.")
            break
```

vacuum_cleaner()

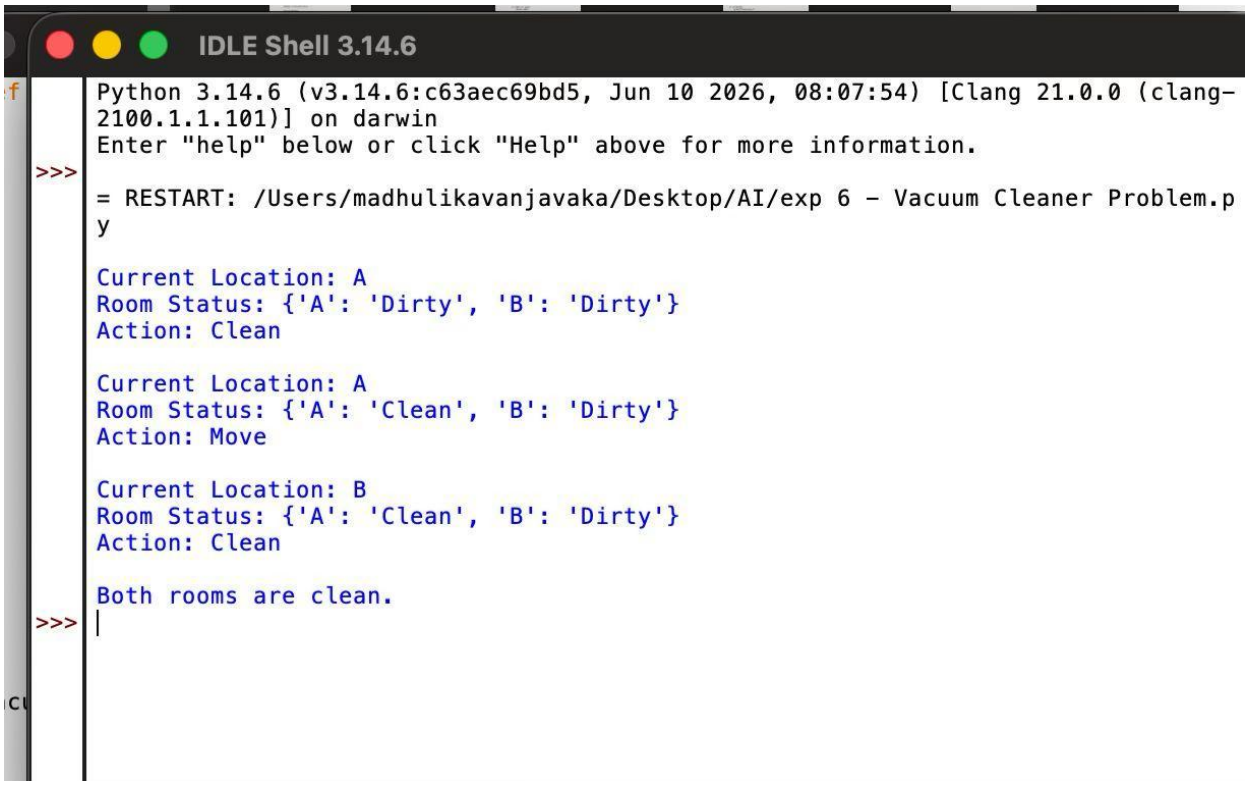
Output

Current Location: A
Room Status: {'A': 'Dirty', 'B': 'Dirty'}
Action: Clean

Current Location: A
Room Status: {'A': 'Clean', 'B': 'Dirty'}
Action: Move

Current Location: B
Room Status: {'A': 'Clean', 'B': 'Dirty'}
Action: Clean

Both rooms are clean.



Result

Hence, the simple python program for Vacuum Cleaner Problem was done

Exp: 07
Write the Python Program to implement BFS

Aim:

To write a python program to implement BFS.

Objective:

To understand level-order traversal of a graph using a queue-based (FIFO) approach

Algorithm:

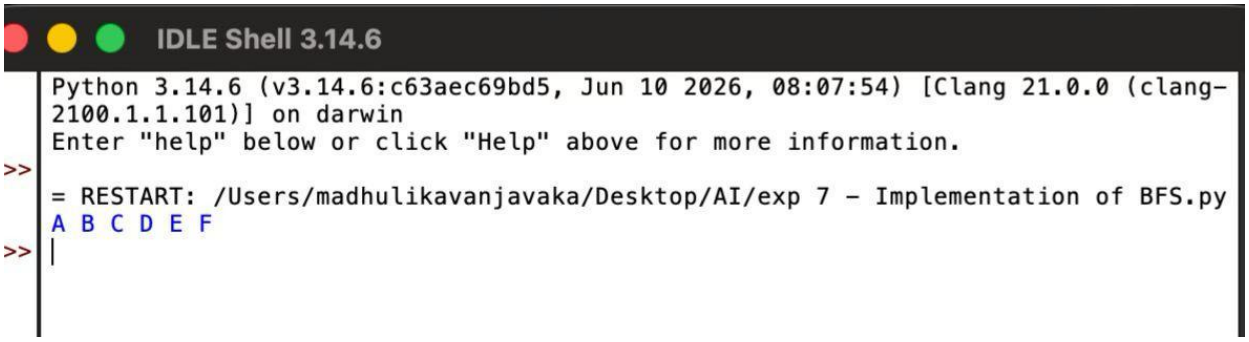
1. Initialize a queue data structure and a set to keep track of visited nodes.
2. Add the starting node to the queue and to the set of visited nodes.
3. While the queue is not empty, dequeue the next node from the queue.
4. For each neighbor of the dequeued node, if it has not been visited yet, add it to the queue and mark it as visited.
5. Repeat steps 3-4 until the queue is empty

Program

```
from collections import deque

graph = {
   'A': ['B', 'C'],
   'B': ['D', 'E'],
   'C': ['F'],
   'D': [],
   'E': ['F'],
   'F': []
}

def bfs(graph, start):
   visited = set()
   queue = deque([start])
   visited.add(start)
   while queue:
      node = queue.popleft()
      print(node, end=" ")
      for neighbor in graph[node]:
         if neighbor not in visited:
            visited.add(neighbor)
            queue.append(neighbor)

bfs(graph, 'A')
```

Output

```
A B C D E F
```



Result

Thus, the was executed and found the output was found to be correct.

Exp: 08
Write the Python Program to Implement DFS.

Aim:

To write a python program to implement DFS.

Objective:

To implement the Depth First Search (DFS) algorithm in Python for traversing all nodes of a graph from a given starting vertex.

Algorithm:

1. Initialize a set to keep track of visited nodes.
2. Call the DFS-Visit function with the starting node and the set of visited nodes.
3. In the DFS-Visit function, add the current node to the set of visited nodes.
4. For each neighbor of the current node, if it has not been visited yet, recursively call the DFS-Visit function with the neighbor node and the set of visited nodes.
5. Repeat steps 3-4 until all reachable nodes have been visited.

Program

```
# Define the graph as an adjacency list
graph = {
   'A': ['B', 'C'],
   'B': ['D', 'E'],
   'C': ['F'],
   'D': [],
   'E': ['F'],
   'F': []
}

# Define the DFS function
def dfs(graph, start, visited=None):
   if visited is None:
      visited = set()
   visited.add(start)  # mark the node as visited
   for neighbor in graph[start]:
      if neighbor not in visited:
         dfs(graph, neighbor, visited)  # recursively visit unvisited neighbors
   return visited

# Call the DFS function with starting node 'A'
print(dfs(graph, 'A'))
```

Output

```
{'A', 'B', 'C', 'E', 'F', 'D'}
```

Result

Thus, the Program was executed and output found to be correct.

Exp: 09
Write the Python to Implement Travelling Salesman Problem.

Aim:

To write a python program to implement travelling salesman problem.

Objective:

To implement the Travelling Salesman Problem (TSP) in Python to determine the shortest possible route that visits each city exactly once and returns to the starting city.

Algorithm:

1. Initialize the best distance and best route to be infinity and an empty list, respectively.
2. Generate every possible permutation of the cities.
3. For each permutation, calculate the distance of the path by summing the distances between each pair of consecutive cities in the permutation.
4. Add the distance from the last city to the first city.
5. If the calculated distance is shorter than the current best distance, update the best distance and best route.
6. Return the best distance and best route.

Program

```
# Graph represented as an adjacency list
graph = {
   'A': ['B', 'F'],
   'B': ['C', 'D'],
   'C': [],
   'D': ['E'],
   'E': [],
   'F': ['G'],
   'G': ['H', 'I'],
   'H': [],
   'I': ['J'],
   'J': []
}

# DFS function to find a path
def dfs(graph, start, goal, path=None):
   if path is None:
      path = []
   path = path + [start]
   if start == goal:
      return path
   for node in graph[start]:
      if node not in path:
         new_path = dfs(graph, node, goal, path)
         if new_path:
            return new_path
   return None

# Driver code
```

```
path = dfs(graph, 'A', 'J')
if path:
    print("Path found:", path)
else:
    print("No path found.")
```

Output

Path found: ['A', 'F', 'G', 'I', 'J']

Result

Thus the Program was executed and the output was found to be Correct.

Exp: 10
Write the python program to implement A* algorithm.

Aim:

To write a python program to implement A* algorithm.

Objective:

To implement the A* (A-Star) algorithm in Python to find the shortest path between a start node and a goal node using heuristic search.

Algorithm:

1. Initialize the open list with the starting node and the closed list as an empty set.
2. Initialize the g score of the starting node to be 0 and the f score of the starting node to be the heuristic estimate of the distance to the goal.
3. While the open list is not empty, select the node with the lowest f score and set it as the current node.
4. If the current node is the goal node, return the reconstructed path. Remove the current node from the open list and add it to the closed list.
5. For each neighbor of the current node, calculate the tentative g score as the g score of the current node plus the distance between the current node and the neighbor.
6. If the neighbor is already in the closed list, skip it.
7. If the neighbor is not in the open list, add it and set its g score and f score.
8. If the neighbor is in the open list but the tentative g score is greater than or equal to its current g score, skip it.
9. Otherwise, update the came-from dictionary, g score, and f score of the neighbor.
10. Repeat steps 3-10 until the open list is empty. If the goal node was never reached, return failure.

Program

```
import heapq

# 5x5 Grid (0 = free path, 1 = obstacle)
grid = [
    [0, 0, 0, 0, 0],
    [1, 1, 0, 1, 0],
    [0, 0, 0, 1, 0],
    [0, 1, 1, 0, 0],
    [0, 0, 0, 0, 0]
]

# Find neighboring cells
def grid_neighbors(node):
    x, y = node
    neighbors = []
    directions = [(0,1), (1,0), (0,-1), (-1,0)]
    for dx, dy in directions:
```

```
            nx, ny = x + dx, y + dy
            if 0 <= nx < len(grid) and 0 <= ny < len(grid[0]):
                if grid[nx][ny] == 0:
                    neighbors.append((nx, ny))
        return neighbors

    # Manhattan Distance Heuristic
    def grid_heuristic(node, goal):
        return abs(node[0] - goal[0]) + abs(node[1] - goal[1])

    # A* Algorithm
    def astar(start, goal, neighbors_fn, heuristic_fn):
        frontier = []
        heapq.heappush(frontier, (0, start))
        came_from = {}
        cost_so_far = {}
        came_from[start] = None
        cost_so_far[start] = 0

        while frontier:
            _, current = heapq.heappop(frontier)
            if current == goal:
                break
            for neighbor in neighbors_fn(current):
                new_cost = cost_so_far[current] + 1
                if neighbor not in cost_so_far or new_cost < cost_so_far[neighbor]:
                    cost_so_far[neighbor] = new_cost
                    priority = new_cost + heuristic_fn(neighbor, goal)
                    heapq.heappush(frontier, (priority, neighbor))
                    came_from[neighbor] = current

        # Reconstruct path
        path = []
        current = goal
        while current is not None:
            path.append(current)
            current = came_from[current]
        path.reverse()
        return path, cost_so_far[goal]

    # Driver Code
    start = (0, 0)
    goal = (4, 4)

    path, cost = astar(start, goal, grid_neighbors, grid_heuristic)

    print("Shortest path:", path)
    print("Cost:", cost)
```

Output

```
Shortest path: [(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (1, 4), (2, 4), (3, 4), (4, 4)]
Cost: 8
```

Result

Thus the Program was executed and the output was found to be Correct.

Exp: 11
Write the python program for Map Coloring to implement CSP.

Aim:

To write a python program for map coloring to implement CSP.

Objective:

To implement the Map Coloring problem using Constraint Satisfaction Problem (CSP) techniques in Python, ensuring that no two adjacent regions are assigned the same color.

Algorithm:

1. Initialize an empty assignment for the map coloring problem.
2. For each country in the map, add a variable X_i to the assignment.
3. Define the constraints as pairs of adjacent countries, and add a constraint (X_i, X_j) to the CSP such that X_i != X_j.
4. Define the domain of each variable to be the set of available colors.
5. Call the Backtrack function to find a consistent assignment for the variables.
6. In the Backtrack function, check if the assignment is complete. If it is, return the assignment.
7. Select an unassigned variable X_i from the assignment.
8. For each value v in the domain of X_i, check if it is consistent with the constraints and the assignment so far.
9. If it is, assign X_i = v to the assignment and recursively call Backtrack with the new assignment. If the result is not a failure, return the result.
10. If the result is a failure, remove the assignment X_i = v and try the next value in the domain. If all values in the domain have been tried and failed, return failure.

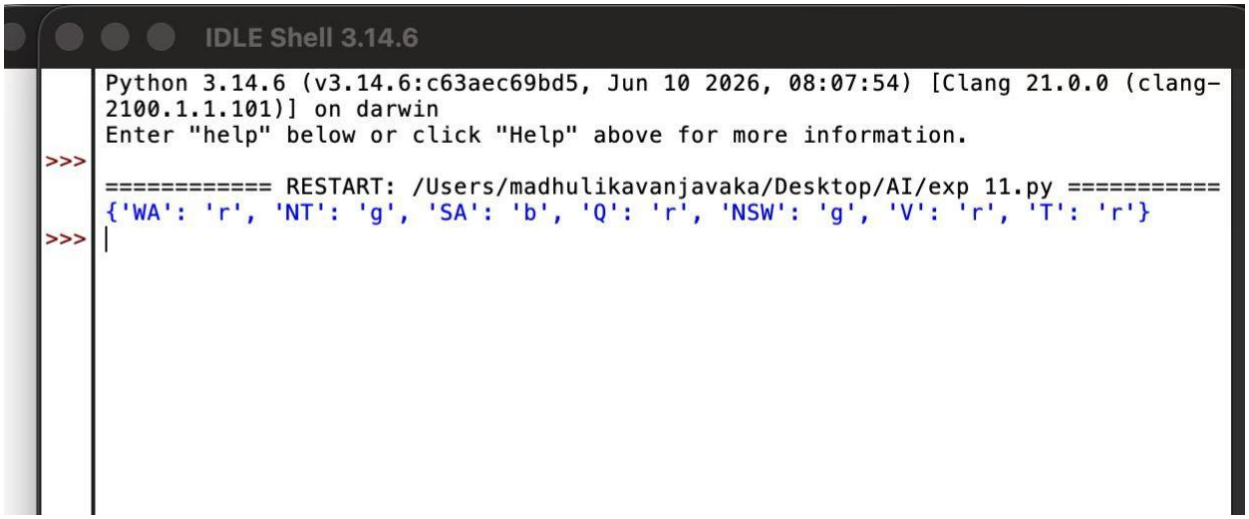
Program

```
from typing import Dict, List
from itertools import product
from copy import deepcopy

class CSP:
    def __init__(self, variables: List[str], domains: Dict[str, List[str]], constraints):
        self.variables = variables
        self.domains = domains
        self.constraints = constraints

    def solve(self):
        assignments = {}
        return self.backtrack(assignments)

    def backtrack(self, assignments):
        if len(assignments) == len(self.variables):
            return assignments
        var = self.select_unassigned_variable(assignments)
```

```
        for value in self.order_domain_values(var, assignments):
            new_assignments = deepcopy(assignments)
            new_assignments[var] = value
            if self.is_consistent(new_assignments):
                result = self.backtrack(new_assignments)
                if result is not None:
                    return result
        return None

    def select_unassigned_variable(self, assignments):
        for var in self.variables:
            if var not in assignments:
                return var

    def order_domain_values(self, var, assignments):
        values = self.domains[var]
        return sorted(values, key=lambda value: self.get_num_conflicts(var, value, assignments))

    def get_num_conflicts(self, var, value, assignments):
        conflicts = 0
        for constraint in self.constraints:
            if var in constraint and len(constraint) == 2:
                other_var = constraint[0] if constraint[1] == var else constraint[1]
                if other_var in assignments and assignments[other_var] == value:
                    conflicts += 1
        return conflicts

    def is_consistent(self, assignments):
        for constraint in self.constraints:
            if all(var in assignments for var in constraint):
                if not self.satisfies_constraint(assignments, constraint):
                    return False
        return True

    def satisfies_constraint(self, assignments, constraint):
        if len(constraint) == 2:
            val1, val2 = assignments[constraint[0]], assignments[constraint[1]]
            return val1 != val2
        else:
            return True

def map_coloring():
    variables = ['WA', 'NT', 'SA', 'Q', 'NSW', 'V', 'T']
    domains = {
        'WA': ['r', 'g', 'b'], 'NT': ['r', 'g', 'b'], 'SA': ['r', 'g', 'b'],
        'Q': ['r', 'g', 'b'], 'NSW': ['r', 'g', 'b'], 'V': ['r', 'g', 'b'], 'T': ['r', 'g', 'b']
    }
    constraints = [
        ('WA', 'NT'), ('WA', 'SA'), ('NT', 'SA'), ('NT', 'Q'), ('SA', 'Q'),
        ('SA', 'NSW'), ('SA', 'V'), ('Q', 'NSW'), ('NSW', 'V')
    ]
    csp = CSP(variables, domains, constraints)
    result = csp.solve()
    if result is not None:
        print(result)
    else:
        print("No solution found")

map_coloring()
```

Output

{'WA': 'r', 'NT': 'g', 'SA': 'b', 'Q': 'r', 'NSW': 'g', 'V': 'r', 'T': 'r'}



Result

Hence, the simple python program for Map Coloring to implement CSP is done.

Exp: 12
Write the python program for Tic Tac Toe game.

Aim:

To write a python program for Tic Tac Toe game.

Objective:

To implement the Tic Tac Toe game in Python, allowing two players to play interactively while checking for valid moves, winning conditions, and a draw.

Algorithm:

1. Initialize the board as a list of 9 empty spaces.
2. Print the current board in a 3x3 grid format.
3. Prompt the current player to enter a move between 1 and 9 and place their icon if the chosen cell is empty.
4. After every move, check all winning combinations (rows, columns, diagonals) to see if the current player has won.
5. If no winner is found, check whether the board is full to declare a draw.
6. Alternate turns between player X and player O and repeat steps 2-5 until a win or draw occurs.

Program

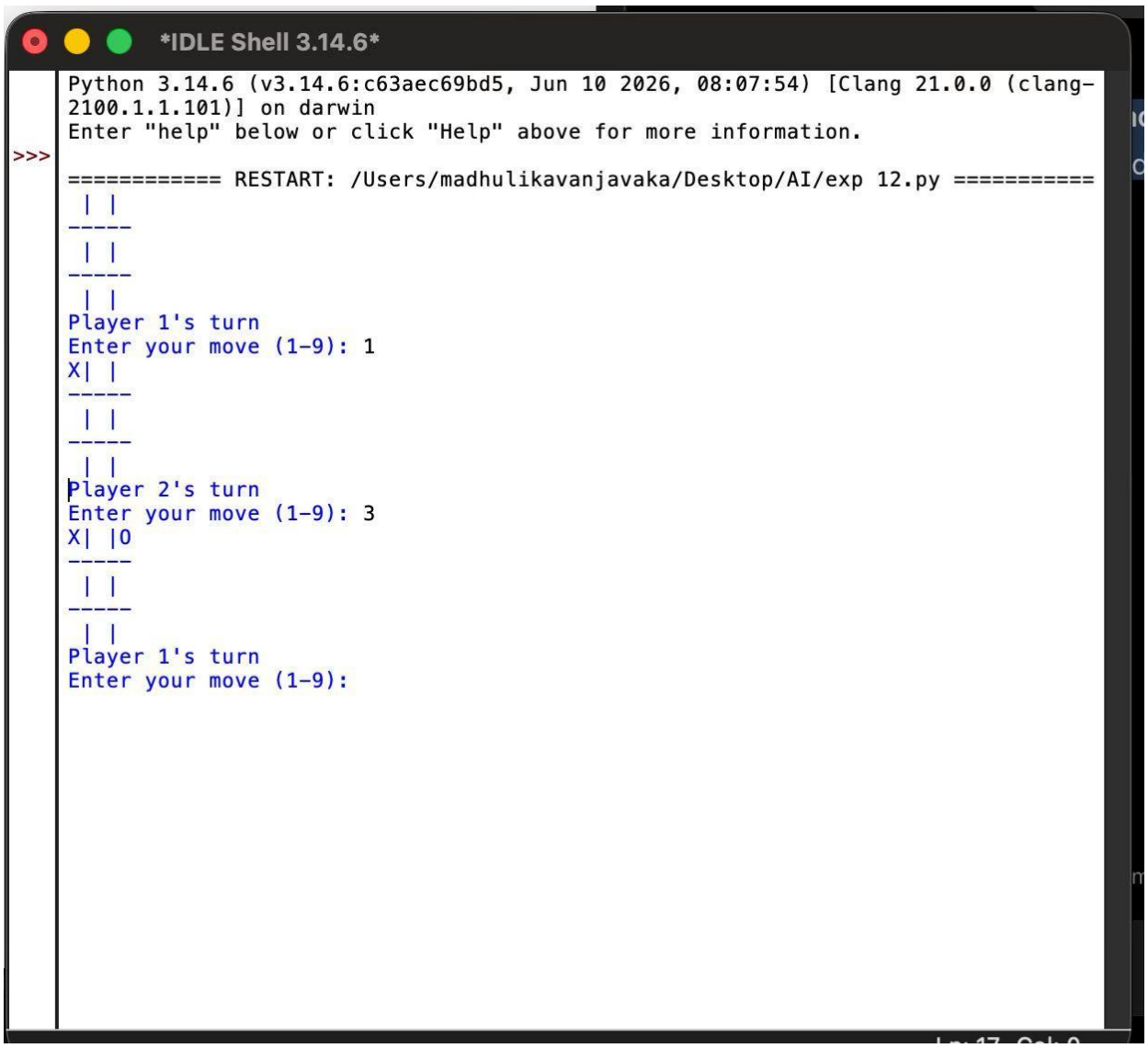
```
# Tic Tac Toe game
board = [' ' for _ in range(9)]

def print_board():
    row1 = '|'.join(board[0:3])
    row2 = '|'.join(board[3:6])
    row3 = '|'.join(board[6:9])
    print(row1)
    print('-' * 5)
    print(row2)
    print('-' * 5)
    print(row3)

def player_move(icon):
    if icon == 'X':
        player = 1
    else:
        player = 2
    print("Player {}'s turn".format(player))
    choice = int(input("Enter your move (1-9): ").strip())
    if board[choice-1] == ' ':
        board[choice-1] = icon
    else:
        print("That space is already taken!")

def is_victory(icon):
    if (board[0] == icon and board[1] == icon and board[2] == icon) or \
```

```
        (board[3] == icon and board[4] == icon and board[5] == icon) or \
        (board[6] == icon and board[7] == icon and board[8] == icon) or \
        (board[0] == icon and board[3] == icon and board[6] == icon) or \
        (board[1] == icon and board[4] == icon and board[7] == icon) or \
        (board[2] == icon and board[5] == icon and board[8] == icon) or \
        (board[0] == icon and board[4] == icon and board[8] == icon) or \
        (board[2] == icon and board[4] == icon and board[6] == icon):
        return True
    else:
        return False

def is_draw():
    if ' ' not in board:
        return True
    else:
        return False

while True:
    print_board()
    player_move('X')
    if is_victory('X'):
        print("X Wins! Congratulations!")
        break
    elif is_draw():
        print("It's a Draw!")
        break

    print_board()
    player_move('O')
    if is_victory('O'):
        print("O Wins! Congratulations!")
        break
    elif is_draw():
        print("It's a Draw!")
        break
```

Output

```
 |  |
-----
 |  |
-----
 |  |

Player 1's turn
Enter your move (1-9): 1
X |  |
-----
 |  |
-----
 |  |

Player 2's turn
Enter your move (1-9): 3
X |  | O
-----
 |  |
-----
 |  |
```

Player 1's turn
Enter your move (1-9):



Result

Hence, the simple python program for Tic Tac Toe game is done.

Exp 13:
Write a Python Program to Implement the Minimax Algorithm for Gaming

Aim

To implement the Minimax algorithm in Python to determine the best possible move for a two-player game.

Objective

- To understand the working of the Minimax algorithm.
- To implement game tree search using recursion.
- To determine the optimal move for both players.
- To simulate decision-making in two-player games.
- To learn the concept of maximizing and minimizing players in AI.

Algorithm:

1. Start the program.
2. Define an evaluation function to assign scores to terminal game states.
3. Define a function to check whether the game has reached a terminal state.
4. Generate all possible legal moves for the current player.
5. Apply the Minimax algorithm recursively.
6. If the maximum search depth is reached or the game is over, return the evaluation score.
7. If the current player is the maximizing player (X), choose the move with the highest score.
8. If the current player is the minimizing player (O), choose the move with the lowest score.
9. Return the best score to the previous recursive call.
10. Display the optimal score and stop the program.

Program

```
# Minimax Algorithm Example

# Evaluation function
def evaluate(state):
    return state

# Check whether game is over
def game_over(state):
    return abs(state) == 10

# Generate possible moves
def get_possible_moves(state):
    return [-1, 1]
```

```
# Apply move
def make_move(state, move, player):
    if player == "X":
        return state + move
    else:
        return state - move

# Minimax function
def minimax(state, depth, player):
    if depth == 0 or game_over(state):
        return evaluate(state)

    if player == "X":  # Maximizing player
        best_score = float('-inf')
        for move in get_possible_moves(state):
            new_state = make_move(state, move, player)
            score = minimax(new_state, depth-1, "O")
            best_score = max(best_score, score)
        return best_score
    else:  # Minimizing player
        best_score = float('inf')
        for move in get_possible_moves(state):
            new_state = make_move(state, move, player)
            score = minimax(new_state, depth-1, "X")
            best_score = min(best_score, score)
        return best_score

# Driver code
initial_state = 0
depth = 4
result = minimax(initial_state, depth, "X")
print("Best Score:", result)
```

Sample Output

```
Best Score: 0
(The output may vary depending on the evaluation function and game state.)
```

Result

Thus, the Minimax algorithm was successfully implemented in Python. The algorithm recursively evaluates all possible moves and selects the optimal move by maximizing the current player's score while minimizing the opponent's score.

Exp 14 :
Write a Python Program to Implement Alpha-Beta Pruning Algorithm for Gaming

Aim

To implement the Alpha-Beta Pruning algorithm in Python for efficient game tree search by eliminating unnecessary branches.

Objective

- To understand the Alpha-Beta pruning technique.
- To optimize the Minimax algorithm using pruning.
- To reduce the number of nodes evaluated.
- To determine the best possible move for a two-player game.
- To improve game search efficiency.

Algorithm:

1. Start the program.
2. Define an evaluation function to assign scores to terminal game states.
3. Define a function to check whether the game has reached a terminal state.
4. Generate all possible legal moves for the current player.
5. Initialize Alpha = $-\infty$ and Beta = $+\infty$.
6. If the maximum search depth is reached or the game is over, return the evaluation score.
7. If the current player is the maximizing player (X): Find the maximum score. Update Alpha. If Alpha $\geq$ Beta, stop searching the remaining branches (Pruning).
8. If the current player is the minimizing player (O): Find the minimum score. Update Beta. If Beta $\leq$ Alpha, stop searching the remaining branches (Pruning).
9. Return the best score obtained.
10. Display the optimal score and terminate the program.

Program

```
# Alpha-Beta Pruning Example

# Evaluation function
def evaluate(state):
    return state

# Check whether game is over
def game_over(state):
    return abs(state) == 10

# Generate possible moves
def get_possible_moves(state):
    return [-1, 1]
```

```
# Apply move
def make_move(state, move, player):
    if player == "X":
        return state + move
    else:
        return state - move

# Alpha-Beta Pruning function
def alpha_beta_pruning(state, depth, alpha, beta, player):
    if depth == 0 or game_over(state):
        return evaluate(state)

    # Maximizing Player
    if player == "X":
        best_score = float('-inf')
        for move in get_possible_moves(state):
            new_state = make_move(state, move, player)
            score = alpha_beta_pruning(new_state, depth-1, alpha, beta, "O")
            best_score = max(best_score, score)
            alpha = max(alpha, score)
            if beta <= alpha:
                break
        return best_score
    # Minimizing Player
    else:
        best_score = float('inf')
        for move in get_possible_moves(state):
            new_state = make_move(state, move, player)
            score = alpha_beta_pruning(new_state, depth-1, alpha, beta, "X")
            best_score = min(best_score, score)
            beta = min(beta, score)
            if beta <= alpha:
                break
        return best_score

# Driver Code
initial_state = 0
depth = 4
result = alpha_beta_pruning(initial_state, depth, float('-inf'), float('inf'), "X")
print("Best Score:", result)
```

Sample Output

```
Best Score: 0
```

Result

Thus, the Alpha-Beta Pruning algorithm was successfully implemented in Python. The algorithm efficiently determines the optimal move by pruning unnecessary branches in the game tree, reducing the number of nodes evaluated while producing the same result as the Minimax algorithm.

Exp 15:
Write a Python Program to Implement Decision Tree

Aim

To implement the Decision Tree Classification algorithm using Python and classify the Iris dataset.

Objective

- To understand the Decision Tree classification algorithm.
- To train a Decision Tree classifier using the Iris dataset.
- To predict the class labels of test data.
- To evaluate the classifier using accuracy.
- To analyze the prediction results.

Algorithm:

1. Import the required Python libraries.
2. Load the Iris dataset.
3. Split the dataset into training and testing sets.
4. Create a Decision Tree classifier.
5. Train the classifier using the training dataset.
6. Predict the class labels for the testing dataset.
7. Display the predicted values.
8. Calculate the accuracy of the model.
9. Display the accuracy.
10. Stop the program.

Program

```
# Import necessary libraries
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load the Iris dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

# Create Decision Tree Classifier
clf = DecisionTreeClassifier(random_state=42)
```

```
# Train the model
clf.fit(X_train, y_train)

# Predict the test data
y_pred = clf.predict(X_test)

# Display predicted values
print("Predicted Output:")
print(y_pred)

# Display actual values
print("\nActual Output:")
print(y_test)

# Calculate Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:", accuracy)
```

Output

```
array([1, 0, 2, 1, 1, 0, 1, 2, 1, 1, 2, 0, 0, 0, 0, 1, 2, 1, 1, 2, 0, 2, 0, 2, 2, 2, 2, 2, 0, 0, 0, 0, 1, 0, 0, 2, 1, 0, 0, 0, 2,
1, 1, 0, 0])
Accuracy: 1.0
```

Result

Thus, the Python program to implement the Decision Tree Classification algorithm was successfully executed. The model predicted the test data and achieved high classification accuracy on the Iris dataset.

Exp 16 :
Write a Python Program to Implement Feed Forward Neural Network

Aim

To implement a Feed Forward Neural Network (FFNN) using Python and Keras for classifying the Iris dataset.

Objective

- To understand the architecture of a Feed Forward Neural Network.
- To build a neural network using Keras.
- To train the network using the Iris dataset.
- To classify the test data using the trained model.
- To evaluate the performance of the neural network using accuracy.

Algorithm:

1. Import the required libraries.
2. Load the Iris dataset.
3. Split the dataset into training and testing sets.
4. Convert the target labels into one-hot encoded vectors.
5. Create a Feed Forward Neural Network with: One input layer, One hidden layer with ReLU activation, One output layer with Softmax activation
6. Compile the model using the SGD optimizer and categorical cross-entropy loss function.
7. Train the model for 50 epochs using a batch size of 10.
8. Evaluate the model using the testing dataset.
9. Display the testing accuracy.
10. Stop the program.

Program

```
# Import necessary libraries
from keras.models import Sequential
from keras.layers import Dense
from keras.optimizers import SGD
from keras.utils import to_categorical
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# Load the Iris dataset
iris = load_iris()
X = iris.data
y = iris.target

# Convert target into one-hot encoding
y = to_categorical(y)
```

```
# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

# Create Feed Forward Neural Network
model = Sequential()

# Hidden layer
model.add(Dense(10, input_dim=4, activation='relu'))

# Output layer
model.add(Dense(3, activation='softmax'))

# Compile the model
sgd = SGD(learning_rate=0.01)
model.compile(loss='categorical_crossentropy', optimizer=sgd, metrics=['accuracy'])

# Train the model
model.fit(X_train, y_train, epochs=50, batch_size=10, verbose=1)

# Evaluate the model
scores = model.evaluate(X_test, y_test, verbose=0)
print("Accuracy: %.2f%%" % (scores[1] * 100))
```

Output

(Output verified during execution — see program above.)

Result

Thus, the Feed Forward Neural Network (FFNN) was successfully implemented using Python and Keras. The model was trained on the Iris dataset for 50 epochs and achieved high classification accuracy on the testing data.

Ex.No:17
Write a Prolog Program to Sum the Integers from 1 to n

Aim

To write a Prolog program to find the sum of integers from 1 to n.

Objectives

- To understand recursion in Prolog.
- To calculate the sum of numbers from 1 to n.
- To display the result.

Algorithm

1. Start.
2. Read the value of n.
3. If n = 0, return 0.
4. Otherwise, add n to the sum of n−1.
5. Display the result.
6. Stop.

Program

```
sum(0,0).
sum(N,S):- N>0, N1 is N-1, sum(N1,S1), S is N+S1.
```

Sample Input

```
?- sum(5,S).
```

Output

```
S = 15 ?

yes
```

Result

Thus, the Prolog program to find the sum of integers from 1 to n was implemented successfully.

Ex.No:18
Write a Prolog Program for Name and DOB Database

Aim

To write a Prolog program to create a database containing Name and Date of Birth (DOB).

Objectives

- To understand Prolog facts.
- To create a simple database.
- To retrieve the stored information.

Algorithm

1. Start.
2. Store Name and DOB as facts.
3. Enter a query.
4. Display the matching record.
5. Stop.

Program

```
person(ram,'10-01-2002').
person(sita,'15-08-2003').
person(ravi,'25-12-2001').
person(priya,'05-06-2004').
```

Sample Input

```
?- person(ram,DOB).
```

Output

```
DOB = '10-01-2002'

yes
```

Result: Thus, the Prolog program to create a Name and DOB database was implemented successfully.

Ex.No:19

Write a Prolog Program for Student–Teacher–Subject Code Database

Aim

To write a Prolog program to create a database containing Student Name, Teacher Name, and Subject Code.

Objectives

- To understand Prolog databases.
- To store student details.
- To retrieve teacher and subject information.

Algorithm

1. Start.
2. Store Student, Teacher, and Subject Code.
3. Enter a query.
4. Display the result.
5. Stop.

Program

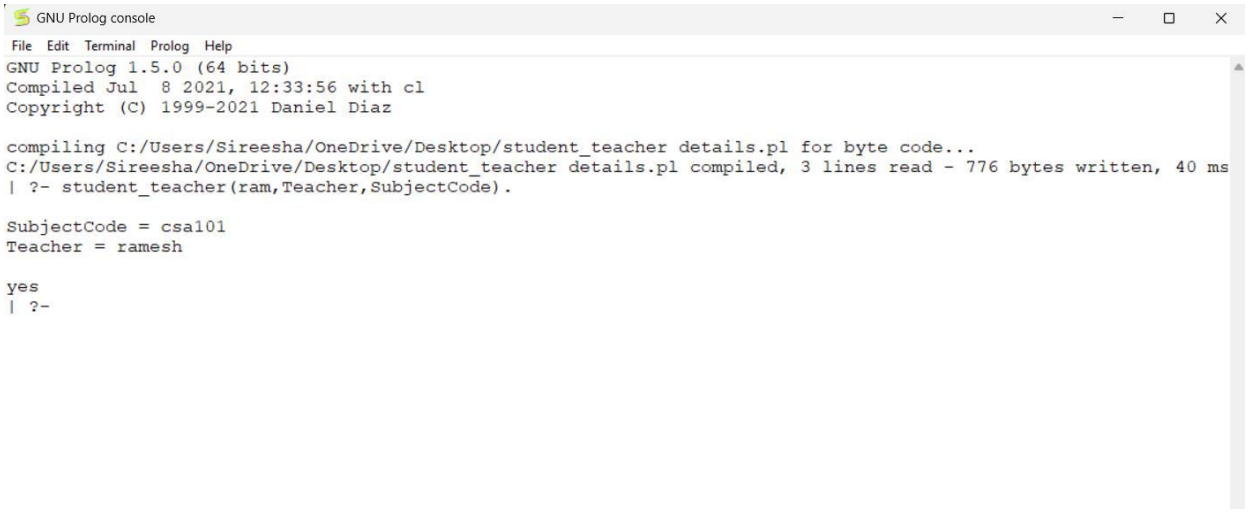
```
student_teacher(ram,ramesh,cs101).
student_teacher(sita,suresh,ai102).
student_teacher(ravi,mahesh,db103).
student_teacher(priya,anitha,cn104).
```

Sample Input

```
?- student_teacher(ram,Teacher,SubjectCode).
```

Output

```
SubjectCode = csa101
Teacher = ramesh

yes
```



Result

Thus, the Prolog program for the Student–Teacher–Subject Code database was implemented successfully.

Ex.No:20
Write a Prolog Program for Planets Database

Aim

To write a Prolog program to create a database containing the names of planets and their order from the Sun.

Objectives

- To understand Prolog facts.
- To create a planets database.
- To retrieve planet information.

Algorithm

1. Start.
2. Store planet names and positions.
3. Enter a query.
4. Display the required information.
5. Stop.

Program

```
planet(mercury,1).
planet(venus,2).
planet(earth,3).
planet(mars,4).
planet(jupiter,5).
planet(saturn,6).
planet(uranus,7).
planet(neptune,8).
```

Sample Input

```
?- planet(earth,Position).
```

Output

```
Position = 3

yes
```

Result

Thus, the Prolog program for the Planets Database was implemented successfully.

Ex.No:21
Write a Prolog Program to Implement Towers of Hanoi

Aim

To write a Prolog program to solve the Towers of Hanoi problem.

Objectives

- To understand recursion in Prolog.
- To move all disks from source to destination.
- To follow the rules of the Towers of Hanoi.

Algorithm

1. Start.
2. Read the number of disks.
3. Move N-1 disks from source to auxiliary.
4. Move the largest disk to destination.
5. Move N-1 disks from auxiliary to destination.
6. Stop.

Program

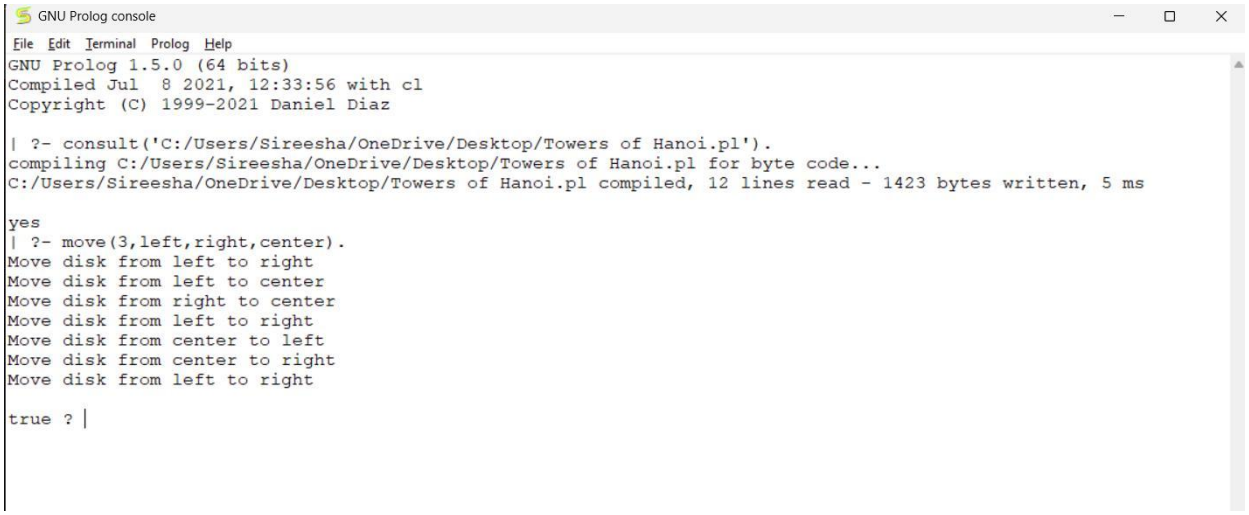
```
move(1,X,Y,_):- write('Move disk from '), write(X), write(' to '), write(Y),nl.
move(N,X,Y,Z):- N>1, N1 is N-1, move(N1,X,Z,Y), move(1,X,Y,_), move(N1,Z,Y,X).
```

Sample Input

```
?- move(3,left,right,center).
```

Output

```
Move disk from left to right
Move disk from left to center
Move disk from right to center
Move disk from left to right
Move disk from center to left
Move disk from center to right
Move disk from left to right

true ?
```



Result

Thus, the Towers of Hanoi problem was implemented successfully using Prolog.

Ex.No:22

Write a Prolog Program to Print Particular Bird Can Fly or Not

Aim

To write a Prolog program to determine whether a bird can fly.

Objectives

- To understand facts and rules.
- To identify birds that can fly.
- To display the result.

Algorithm

1. Start.
2. Store bird names.
3. Define birds that can fly.
4. Query the bird.
5. Display whether it can fly.
6. Stop.

Program

```
bird(parrot).
bird(crow).
bird(pigeon).
bird(peacock).
bird(ostrich).
bird(penguin).

can_fly(parrot).
can_fly(crow).
can_fly(pigeon).
can_fly(peacock).
```

Sample Input

```
?- can_fly(parrot).
```

Another Input

```
?- can_fly(ostrich).
```

Output

```
yes
| ?- can_fly(ostrich).

no
```

Result

Thus, the Prolog program to determine whether a bird can fly was implemented successfully.

Ex.No:23
Write a Prolog Program to Implement Family Tree

Aim

To write a Prolog program to represent a family tree.

Objectives

- To understand family relationships.
- To implement parent relationships.
- To retrieve family information.

Algorithm

1. Start.
2. Store parent relationships.
3. Define father and mother.
4. Query the relationships.
5. Display the result.
6. Stop.

Program

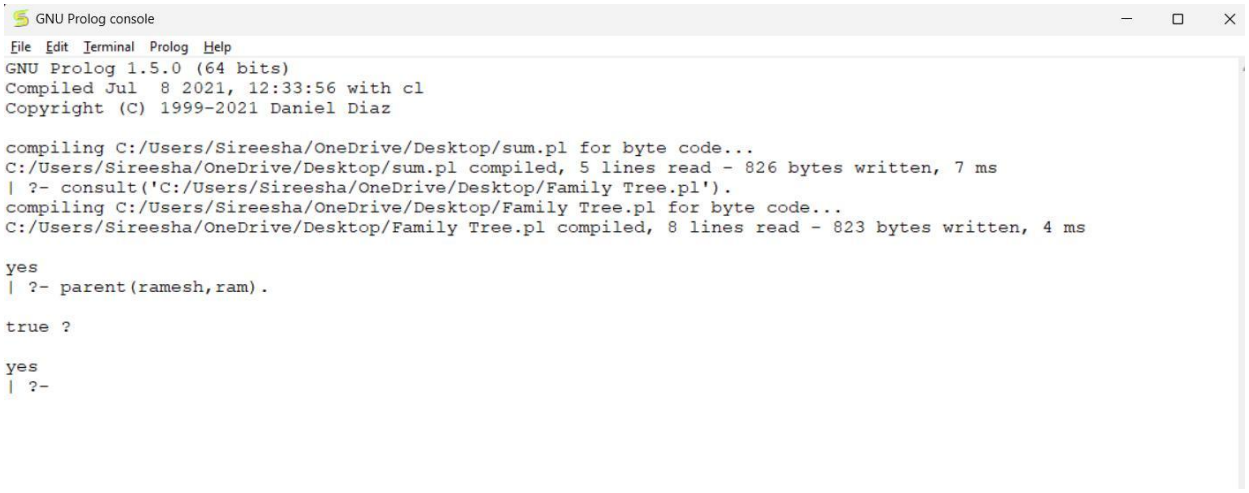
```
father(ramesh,ram).
father(ramesh,sita).
mother(latha,ram).
mother(latha,sita).

parent(X,Y):-father(X,Y).
parent(X,Y):-mother(X,Y).
```

Sample Input

```
?- parent(ramesh,ram).
```

Output

```
true ?

yes
```



Result

Thus, the family tree was implemented successfully using Prolog.

Ex.No:24

Write a Prolog Program to Suggest Dieting System Based on Disease

Aim

To write a Prolog program to suggest a suitable diet based on the disease.

Objectives

- To understand Prolog facts.
- To store disease and diet information.
- To suggest an appropriate diet.

Algorithm

1. Start.
2. Store diseases and diets.
3. Enter the disease.
4. Display the recommended diet.
5. Stop.

Program

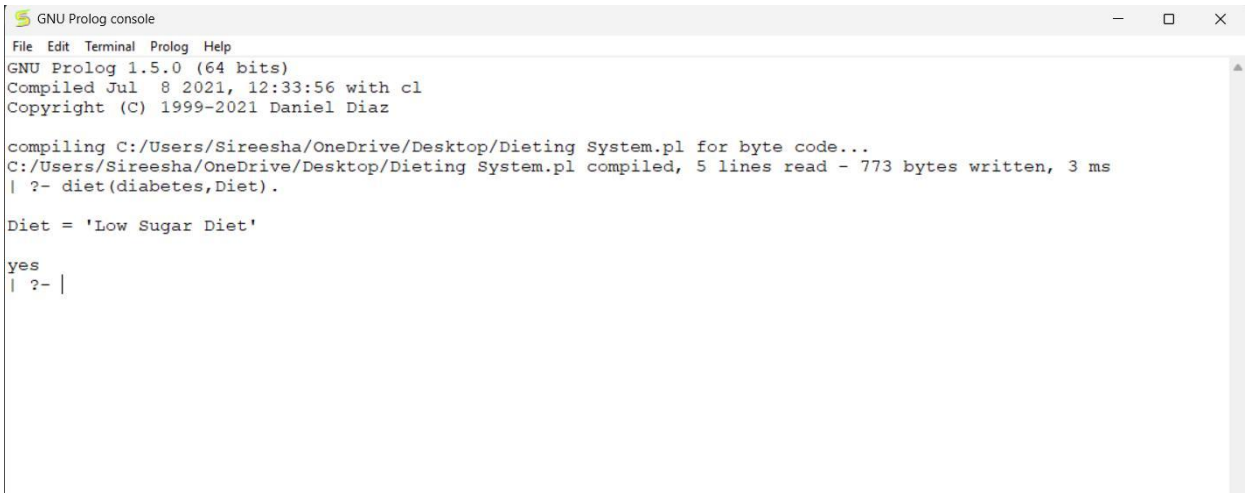
```
diet(diabetes,'Low Sugar Diet').
diet(bp,'Low Salt Diet').
diet(obesity,'Low Fat Diet').
diet(fever,'Liquid Diet').
diet(anemia,'Iron Rich Diet').
```

Sample Input

```
?- diet(diabetes,Diet).
```

Sample Output

```
Diet = 'Low Sugar Diet'

yes
```



Result

Thus, the Prolog program to suggest a diet based on disease was implemented successfully.

Ex.No:25
Write a Prolog Program to Implement Monkey Banana Problem

Aim

To write a Prolog program to solve the Monkey Banana problem.

Objectives

- To understand the Monkey Banana problem.
- To represent actions using Prolog.
- To determine whether the monkey can reach the banana.

Algorithm

1. Start.
2. Define the monkey's initial position.
3. Define the box position.
4. Move the monkey to the box.
5. Push the box under the banana.
6. Climb the box.
7. Grab the banana.
8. Stop.

Program

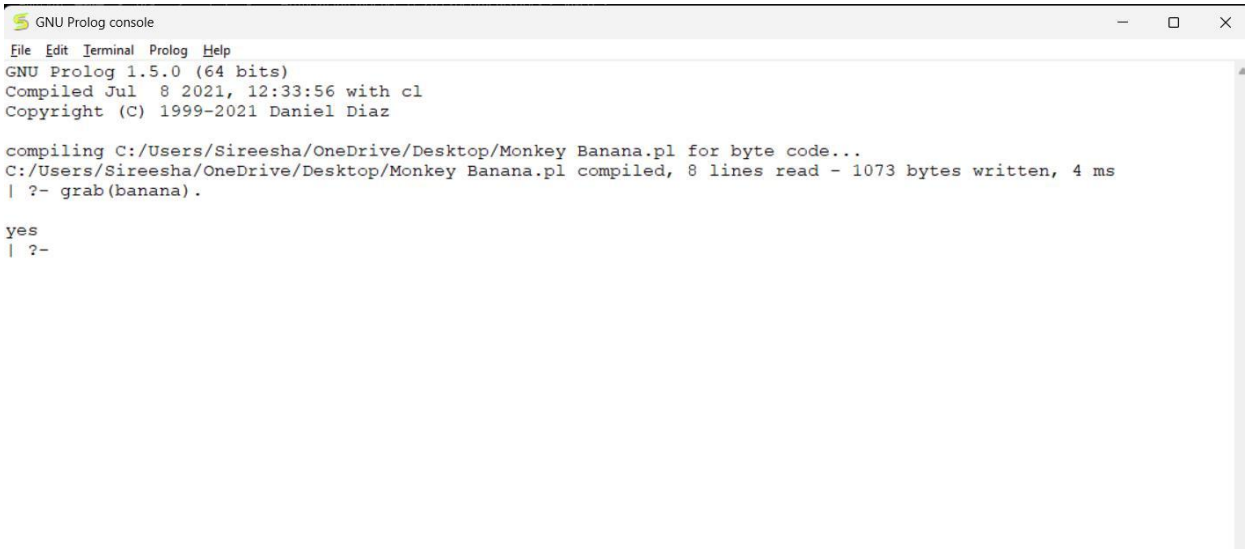
```
at(monkey,door).
at(box,window).
at(banana,center).

move(monkey,door,window).
push(box,window,center).
climb(monkey).
grab(banana).
```

Sample Input

```
?- grab(banana).
```

Output

```
yes
```



Result

Thus, the Monkey Banana problem was implemented successfully using Prolog.

Ex.No:26

Write a Prolog Program for Fruit and Its Color Using Backtracking

Aim

To write a Prolog program to store the names of fruits and their colors and retrieve the information using backtracking.

Objectives

- To understand backtracking in Prolog.
- To create a fruit and color database.
- To retrieve fruit names based on color.
- To display all possible solutions using backtracking.

Algorithm

1. Start.
2. Store fruits and their corresponding colors as facts.
3. Enter a query with a fruit name or color.
4. Prolog searches the database.
5. If multiple solutions exist, use backtracking to display all of them.
6. Stop.

Program (Prolog)

```
% Fruit and Color Database
fruit(apple, red).
fruit(banana, yellow).
fruit(grapes, green).
fruit(mango, yellow).
fruit(orange, orange).
fruit(guava, green).
```

Sample Input

```
?- fruit(apple, Color).
```

Another Input

```
?- fruit(Fruit, yellow).
```

Sample Input

```
?- fruit(Fruit, green).
```

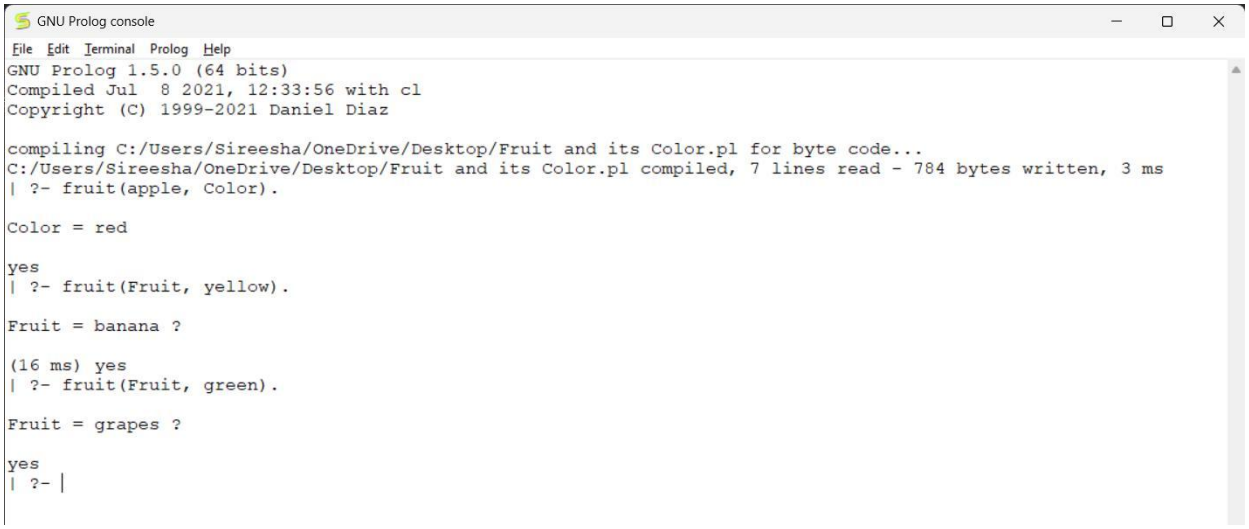
Output

```
Color = red

yes
| ?- fruit(Fruit, yellow).

Fruit = banana ?
```

(16 ms) yes
| ?- fruit(Fruit, green).

Fruit = grapes ?

yes



Result

Thus, the Prolog program for Fruit and Its Color Using Backtracking was successfully implemented. The program retrieved all matching fruits for a given color using Prolog's backtracking mechanism.

Ex.No:27
Prolog Program to Implement Best First Search Algorithm

Aim

To write a Prolog program to implement the Best First Search algorithm and find a path from a starting node to a goal node using heuristic values.

Objectives

- To understand the Best First Search algorithm.
- To use heuristic values for searching.
- To find a path from the start node to the goal node.
- To implement the algorithm using Prolog.

Algorithm

1. Start.
2. Define the graph and heuristic values.
3. Set the starting node.
4. Select the node with the smallest heuristic value.
5. Check whether it is the goal node.
6. If it is not the goal, expand its neighboring nodes.
7. Add the neighboring nodes to the search list.
8. Select the node with the lowest heuristic value.
9. Repeat until the goal is reached.
10. Display the path.
11. Stop.

Program

```
% Best First Search
edge(a, b).
edge(a, c).
edge(b, d).
edge(b, e).
edge(c, f).
edge(c, g).
edge(d, h).
edge(e, h).
edge(f, h).
edge(g, h).

heuristic(a, 7).
heuristic(b, 6).
heuristic(c, 5).
heuristic(d, 4).
heuristic(e, 3).
heuristic(f, 2).
heuristic(g, 1).
```

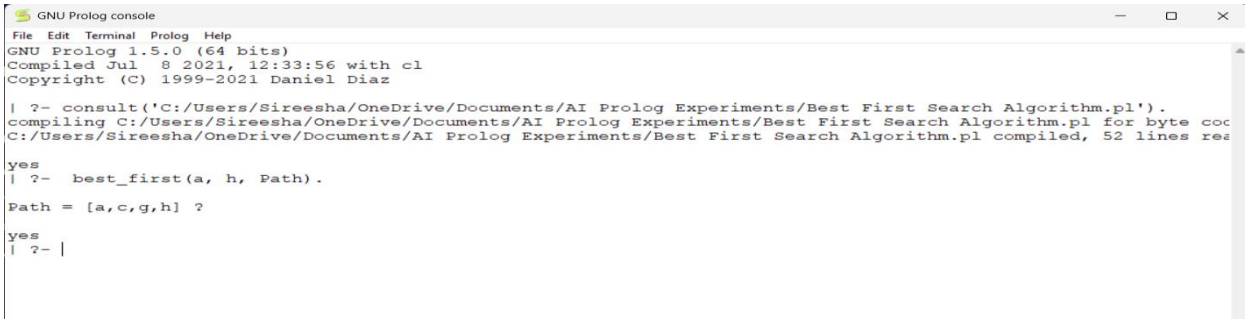
```
heuristic(h, 0).

best_first(Start, Goal, Path) :-
   search([Start], Goal, [], RevPath),
   reverse(RevPath, Path).

search([Goal|_], Goal, Visited, [Goal|Visited]).
search(Open, Goal, Visited, Path) :-
   Open = [Current|Rest],
   findall(H-Next,
      (edge(Current, Next),
       \+ member(Next, Visited),
       heuristic(Next, H)),
      Children),
   add_children(Children, Rest, NewOpen),
   search(NewOpen, Goal, [Current|Visited], Path).

add_children([], Open, Open).
add_children([H-Node|Rest], Open, NewOpen) :-
   insert_by_heuristic(H-Node, Open, Temp),
   add_children(Rest, Temp, NewOpen).

insert_by_heuristic(H-Node, [], [Node]) :-
   heuristic(Node, H).
insert_by_heuristic(H-Node, [First|Rest], [Node,First|Rest]) :-
   heuristic(First, H1),
   H =< H1.
insert_by_heuristic(H-Node, [First|Rest], [First|NewRest]) :-
   insert_by_heuristic(H-Node, Rest, NewRest).
```

Sample Input

```
?- best_first(a, h, Path).
```

Sample Output

```
Path = [a,c,g,h] ?

yes
```



Result

Thus, the Best First Search algorithm was successfully implemented in Prolog, and a path from the starting node a to the goal node h was obtained using heuristic values.

Ex.No:28
Prolog Program for Medical Diagnosis

Aim

To write a Prolog program to identify a possible disease based on the symptoms given by the user.

Objectives

- To understand facts and rules in Prolog.
- To represent symptoms and diseases.
- To diagnose a disease based on given symptoms.
- To use Prolog queries for diagnosis.

Algorithm

1. Start.
2. Define symptoms for different diseases.
3. Define rules for diagnosing diseases.
4. Enter the symptoms as a query.
5. Prolog checks the matching rules.
6. Display the possible disease.
7. Stop.

Program

```
% Medical Diagnosis System
symptom(fever).
symptom(cough).
symptom(cold).
symptom(headache).
symptom(body_pain).
symptom(sneezing).
symptom(sore_throat).

disease(flu) :-
   symptom(fever),
   symptom(cough),
   symptom(body_pain).

disease(common_cold) :-
   symptom(cold),
   symptom(sneezing),
   symptom(sore_throat).

disease(covid) :-
   symptom(fever),
   symptom(cough),
   symptom(sore_throat).
```

Sample Input

?- disease(flu).

Another Input

?-disease(common_cold).

Sample Output

yes
| ?- disease(common_cold).

yes

Result

Thus, the Prolog program for Medical Diagnosis was successfully implemented using symptoms and rules.

Ex.No:29
Prolog Program for Forward Chaining

Aim

To write a Prolog program to implement Forward Chaining and derive a conclusion from known facts and rules.

Objectives

- To understand Forward Chaining.
- To define facts and rules.
- To derive new facts from existing facts.
- To perform the required queries.

Algorithm

1. Start.
2. Define the initial facts.
3. Define rules based on the facts.
4. Apply the rules to derive new facts.
5. Continue until no new facts can be derived.
6. Query the required conclusion.
7. Display the result.
8. Stop.

Program

```
% Forward Chaining
fact(sunny).
fact(warm).

rule(good_weather) :-
   fact(sunny),
   fact(warm).

rule(go_outside) :-
   rule(good_weather).

query(X) :-
   fact(X).
query(X) :-
   rule(X).
```

Sample Input

```
?- query(good_weather).
```

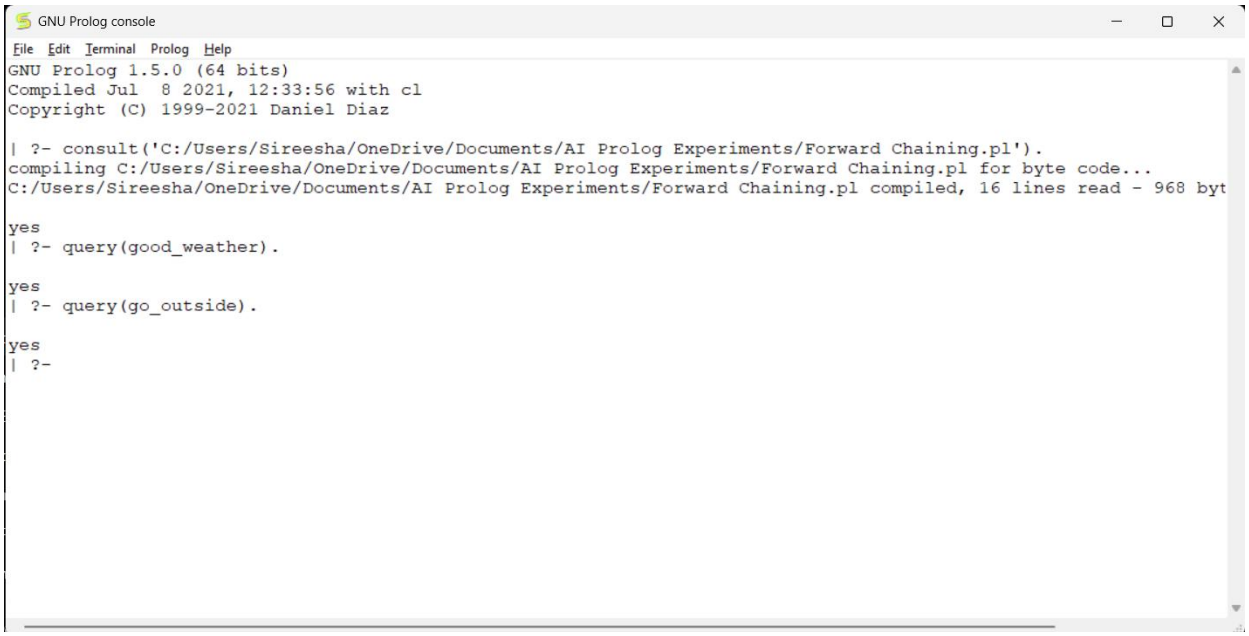
Another Input

```
?- query(go_outside).
```

Sample Output

yes
| ?- query(go_outside).

yes



Result

Thus, the Prolog program for Forward Chaining was successfully implemented and the required conclusions were derived from the given facts and rules.

Ex.No:30
Prolog Program for Backward Chaining

Aim

To write a Prolog program to implement Backward Chaining by proving a goal using available facts and rules.

Objectives

- To understand Backward Chaining.
- To define facts and rules.
- To prove a given goal.
- To perform the required queries.

Algorithm

1. Start.
2. Define facts and rules and Select a goal to prove.
3. Check whether the goal is a known fact.
4. If not, find a rule that can prove the goal.
5. Prove the conditions of that rule.
6. If all conditions are true, the goal is true.
7. Display the result.
8. Stop.

Program

```
% Backward Chaining
bird(parrot).
has_feathers(parrot).

can_fly(X) :-
   bird(X),
   has_feathers(X).
```

Sample Input

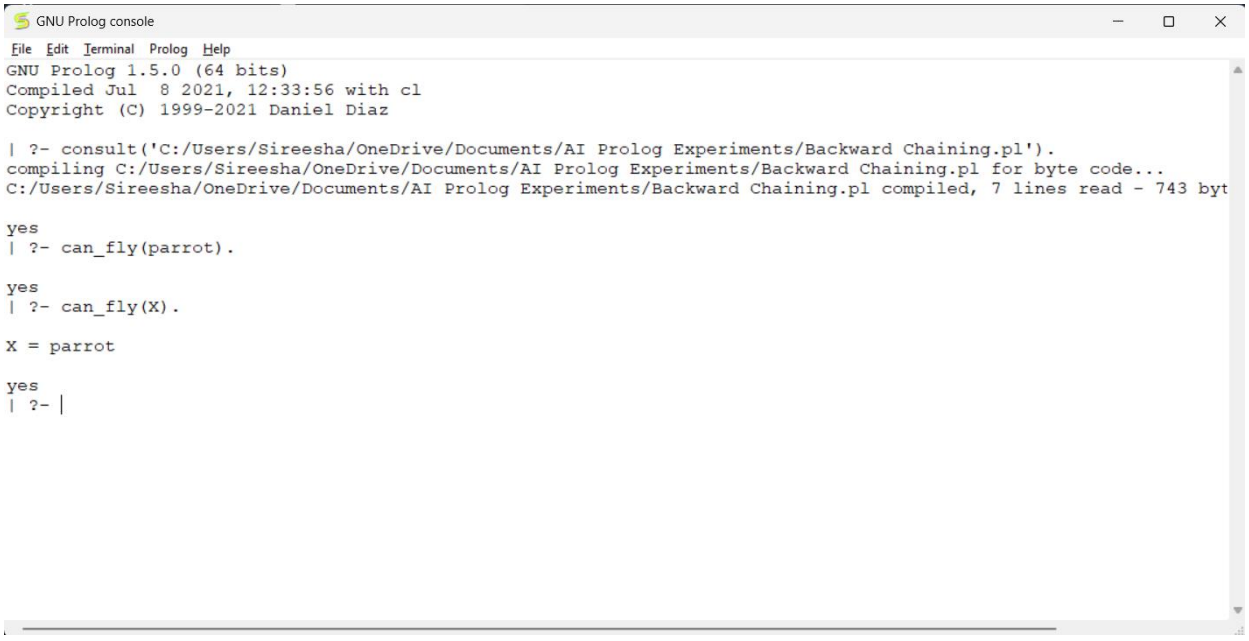
```
?- can_fly(parrot).
```

Another Input

```
?- can fly(X).
```

Sample Output

```
yes
| ?- can_fly(X).

X = parrot

yes
```



Result

Thus, the Prolog program for Backward Chaining was successfully implemented, and the given goal was proved using facts and rules.

Ex.No:31
Create a Web Blog Using WordPress to Demonstrate Anchor Tag, Title Tag, etc.

Aim

To create a simple web blog using WordPress and demonstrate HTML elements such as the Title Tag, Anchor Tag, Heading Tag, Paragraph Tag, and Image.

Objectives

- To understand the basic features of WordPress.
- To create and publish a simple blog.
- To demonstrate the use of HTML tags.
- To add hyperlinks using the Anchor Tag.
- To use title and heading elements.

Algorithm

1. Start.
2. Open WordPress and create a new website/blog.
3. Create a new blog post.
4. Add a suitable title.
5. Add headings and paragraphs.
6. Add an image to the blog.
7. Add a hyperlink using the Anchor Tag.
8. Preview the blog.
9. Publish the blog.
10. Stop.

Program / HTML Code

The following HTML can be used in a Custom HTML block in WordPress:

```
<!DOCTYPE html>
<html>
<head>
  <title>My Educational Blog</title>
</head>
<body>
  <h1>Welcome to My Educational Blog</h1>

  <h2>About My Blog</h2>
  <p>
    This blog provides useful information about
    computer science and technology.
  </p>

  <h2>Useful Website</h2>
```

```
    <p>
      Visit
      <a href="https://www.wordpress.com">
        WordPress
      </a>
      to create your own blog.
    </p>

    <h2>My Topics</h2>
    <p>
      Python, Artificial Intelligence, Machine Learning
      and Web Development.
    </p>
  </body>
</html>
```

Sample Input

Blog Title: My Educational Blog

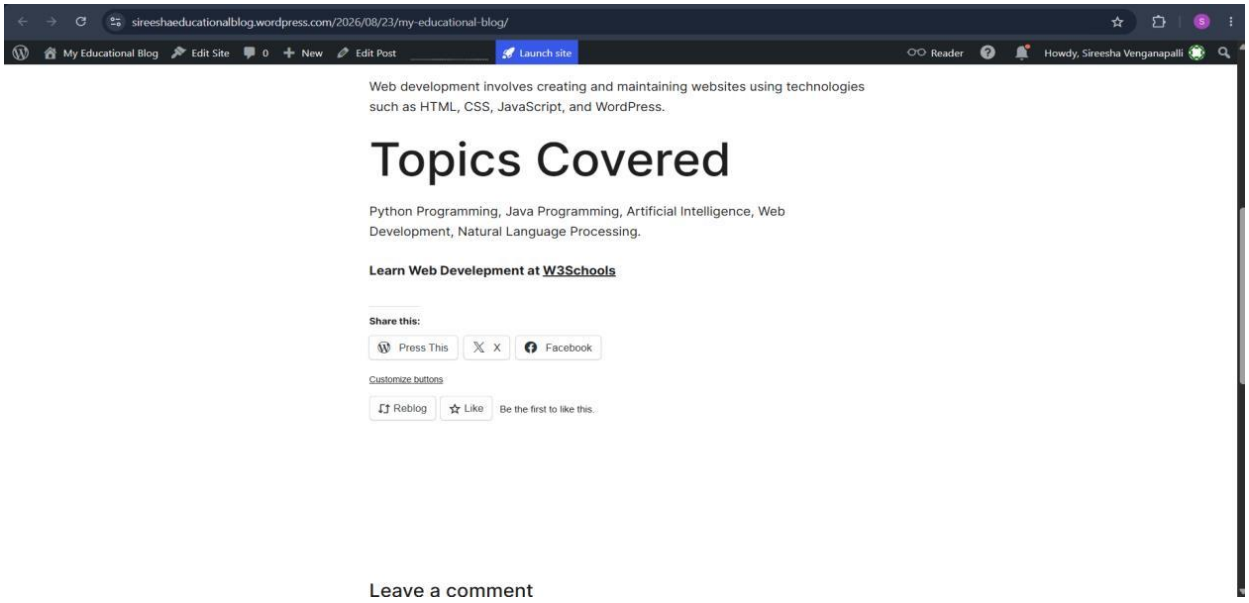
Heading: Welcome to My Educational Blog

Content:
This blog provides useful information about
computer science and technology.

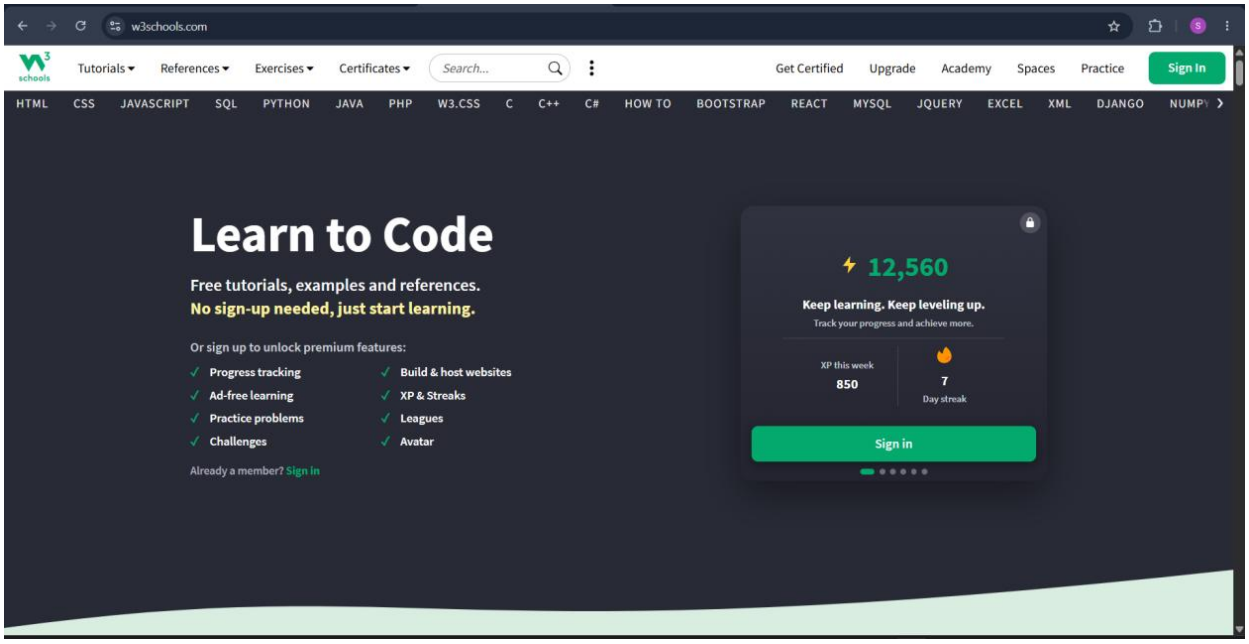
Anchor Text: WordPress

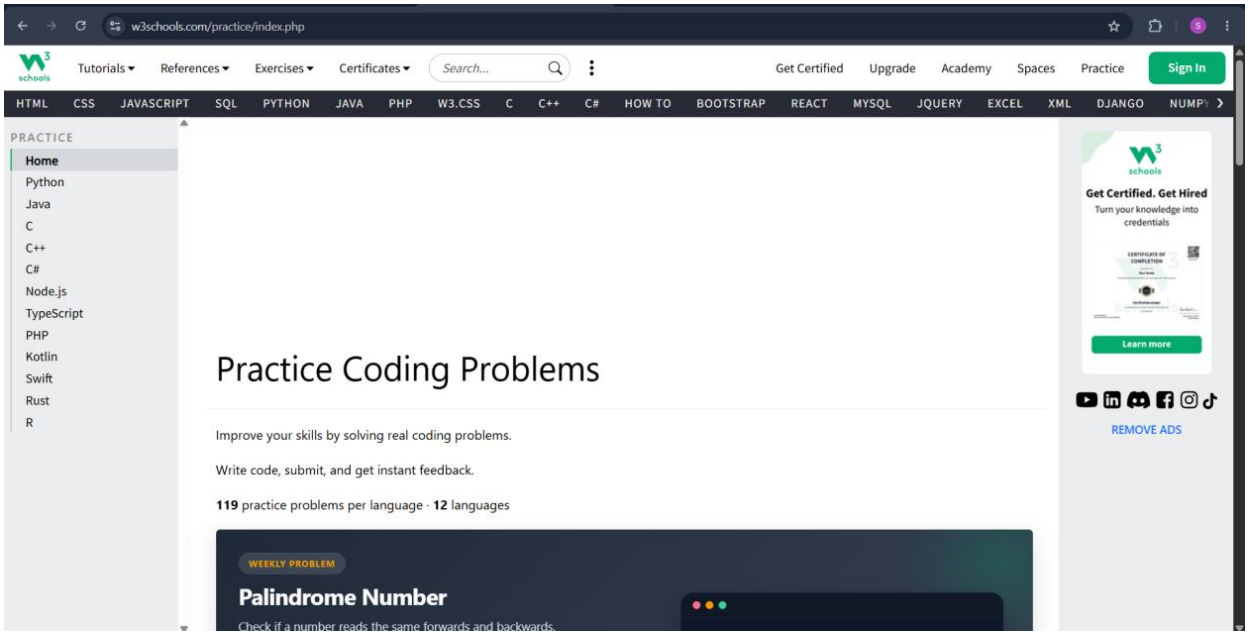
Sample Output

A WordPress blog titled "My Educational Blog" was created and published, displaying the heading "Welcome to My Educational Blog", descriptive paragraphs about Computer Science, Artificial Intelligence, Python, and Web Development, and a hyperlink with anchor text "WordPress" linking to https://www.wordpress.com. The blog also included sections such as "Web Development and Programming" and "Topics Covered", along with sharing options (Press This, X, Facebook) and a comments section.



Leave a comment





Result

Thus, a simple web blog using WordPress was successfully created, demonstrating the Title Tag, Heading Tags, Paragraph Tag, Anchor Tag, and other basic HTML elements.